Deadlock Detection

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;

    // 1. Input parameters
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the number of resources: ");
    scanf("%d", &m);

    int alloc[n][m];
    int request[n][m];
    int avail[m];

    // 2. Input Allocation Matrix
    printf("Enter the allocation matrix:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    // 3. Input Request Matrix
    printf("Enter the request matrix:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (int j = 0; j < m; j++) {
            scanf("%d", &request[i][j]);
        }
    }
```

```
// 4. Input Available Resources
printf("Enter the available resources: ");
for (int j = 0; j < m; j++) {
   scanf("%d", &avail[j]);
}

// 5. Initialize structures for Detection Algorithm
int work[m];
for (int j = 0; j < m; j++) {
   work[j] = avail[j];
}

bool finish[n];
for (int i = 0; i < n; i++) {
   // Condition 1: If allocation is 0 for all resources, finish[i] = true
   bool holds_resources = false;
   for (int j = 0; j < m; j++) {
      if (alloc[i][j] > 0) {
         holds_resources = true;
         break;
      }
   }

   if (!holds_resources) {
      finish[i] = true;
   } else {
      finish[i] = false;
   }
}

int safe_sequence[n];
int count = 0;

// 6. Detection Loop
for (int k = 0; k < n; k++) {
   for (int i = 0; i < n; i++) {
      if (!finish[i]) {
```

```
            bool can_execute = true;

            // Check if current Request <= Work
            for (int j = 0; j < m; j++) {
                if (request[i][j] > work[j]) {
                    can_execute = false;
                    break;
                }
            }

            // If requests can be fulfilled, assume process completes and returns resources
            if (can_execute) {
                for (int j = 0; j < m; j++) {
                    work[j] += alloc[i][j];
                }
                finish[i] = true;
                safe_sequence[count++] = i;
            }
        }
    }
}

// 7. Evaluate results to match requested screen output format
bool deadlocked = false;
for (int i = 0; i < n; i++) {
    // If an allocation-holding process couldn't finish, deadlock exists
    if (!finish[i]) {
        deadlocked = true;
        break;
    }
}

if (!deadlocked) {
    printf("System is in safe state.\n");
    printf("Safe Sequence is:");
    for (int i = 0; i < count; i++) {
        printf(" P%d", safe_sequence[i]);
```

```
        }
        printf("\n");
    } else {
        printf("System is in a deadlocked state.\n");
        printf("Deadlocked processes are:");
        for (int i = 0; i < n; i++) {
            if (!finish[i]) {
                printf(" P%d", i);
            }
        }
        printf("\n");
    }

    return 0;
}
```

OUTPUT: